

WHAT IT IS

Analytics File Processor is a multi-page Streamlit app for the Eagle Pace mutual fund analytics team. It lets users upload tabular exports, transform them in-session, and export the results in several business-ready formats.

WHO IT'S FOR

Primary user/persona: an internal analyst on the Eagle Pace mutual fund analytics team working with CSV, TXT, XLSX, and Oracle-sourced datasets.

HOW TO RUN

- Install dependencies: `pip install -r app/requirements.txt`
- Ensure `app/.streamlit/secrets.toml` contains `APP_PASSWORD_HASH`; generate it with the `repo's hashlib` command if needed.
- Start the app: `cd app`
- Run: `streamlit run app.py`

REPO NOTES

- Authentication is required on every page via `utils/auth.py`.
- No test framework or lint framework is configured.

WHAT IT DOES

- Uploads CSV, TXT, and XLSX files; supports multi-file sessions and Excel sheet selection.
- Detects delimiter and encoding, previews data, and infers column types at upload time.
- Merges loaded files by stacking rows or joining on a shared key.
- Applies per-file transforms: rename/reorder/include columns, multi-column sort, row filters, and type/null handling.
- Shows data-quality signals in a sidebar, including null counts, duplicate detection, and 3-sigma outliers.
- Saves and reuses column templates from a local JSON store.
- Exports active data as CSV/TXT, Excel, Power BI-ready CSV, Oracle INSERT SQL, PDF, or an Outlook draft email; Oracle query results can also be sent into the transform pipeline.

HOW IT WORKS

- UI surface: Streamlit shell in `app.py` plus pages for Upload, Transform, Export, Templates, and Oracle Query.
- State model: `st.session_state["files"][filename]` stores `original_df`, `active_df`, `pipeline`, `meta`, and `transform` working state for each loaded file.
- Core services: `utils/transforms.py` handles casting, filtering, and replay logic; `utils/export.py` builds Excel, SQL, PDF, and Outlook outputs; `utils/dq.py` renders the data-quality sidebar; `utils/audit.py` records user actions.
- Data flow: Upload registers each file -> Transform recomputes `active_df` from typed/base data, sort, and column ops -> Export writes downloads or files to disk.
- Persistence and integration: Templates are saved to `~/analytics_processor_templates.json`, and the Oracle Query page can execute SQL, export result sets, or push them back into the same transform/export flow.